

Contenido

1. El motor V8 de Google y la aparición de Node.js	1
2. Proyectos con node	2
3. El objeto global	6
3.1. Funciones.....	6
3.2. Propiedades.....	9

El propósito del módulo DI es crear interfaces gráficas de usuario para entornos de escritorio. Esto se puede hacer con lenguajes de programación como java o c# pero también se puede hacer usando tecnologías web para generar aplicaciones de escritorio. En este último caso, usaremos node, que nos permite ejecutar javascript directamente sin usar un navegador. Este tema es una introducción a node y proyectos con node.

1. El motor V8 de Google y la aparición de Node.js

Hasta 2008, JavaScript vivía casi encerrado en el navegador, limitado a dar dinamismo a las páginas web. Con la llegada de Google Chrome y su motor V8, todo cambió: por primera vez JavaScript podía ejecutarse con un rendimiento muy alto, gracias a que V8 no solo interpreta el código, sino que lo compila a código máquina en tiempo real (JIT).

Cada navegador tiene su propio motor (Safari usa JavaScriptCore, Firefox SpiderMonkey, Edge Chakra...), pero V8 destacó porque se diseñó desde el inicio pensando en velocidad y portabilidad. Lo más revolucionario fue que este motor es independiente del navegador: se puede usar en cualquier entorno para ejecutar JavaScript.

Ahí entra en juego Node.js (2009), que aprovecha V8 para ejecutar JavaScript directamente en el sistema operativo, sin necesidad de abrir un navegador. Esto convirtió a JavaScript en un lenguaje de propósito general, válido también para el backend y para aplicaciones fuera de la web.

Concretamente de node tenemos:

- Mecanismos de acceso al sistema de archivos, especialmente útil para leer archivos de texto, o subir imágenes al servidor, por poner dos ejemplos.
- Mecanismos de conexión con bases de datos.
- Comunicarse a través de Internet (el estándar ECMAScript no tiene estos elementos para Javascript).
- Aceptar solicitudes de clientes y enviar respuestas a esas solicitudes.

Node se puede descargar de la web: <https://nodejs.org/es/>

Podemos ejecutar archivos JavaScript sin necesidad de un navegador, para ejecutar un archivo, desde la línea de comando, escriba: **node nombre.js**

IMPORTANTE!!

Si ya teníais node instalado, es posible que esté desactualizado (por ejemplo, **en clase**). Para actualizarlo ejecutar lo siguiente en la terminal de visual studio code:

#Vemos la versión que tenemos:

node -v

si es < 14, actualiza con nvm. Descarga el script de instalación de nvm (Node Version Manager) desde GitHub y lo ejecuta con bash:

curl -o- <https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh> | bash

cierra y abre terminal

#Usa nvm para instalar y usar la última versión LTS (Long Term Support) de Node.js.

nvm install --lts

nvm use --lts

comprueba que es ≥ 18

node -v

2. Proyectos con node

NPM

npm es el sistema de administración de paquetes predeterminado para Node.js

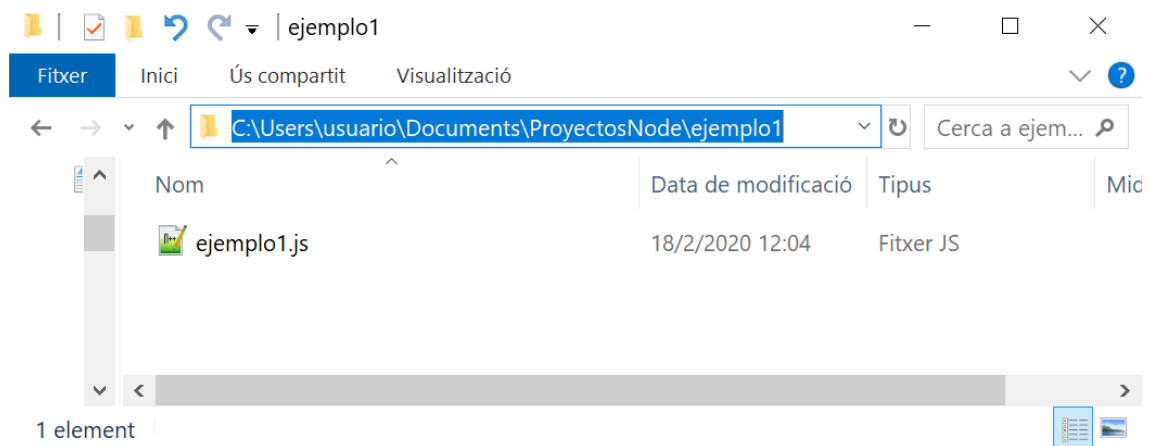
Proyectos en Node

Lo habitual cuando se trabaja con node es tener una carpeta que contendrá el proyecto. Para iniciar un proyecto usaremos npm:

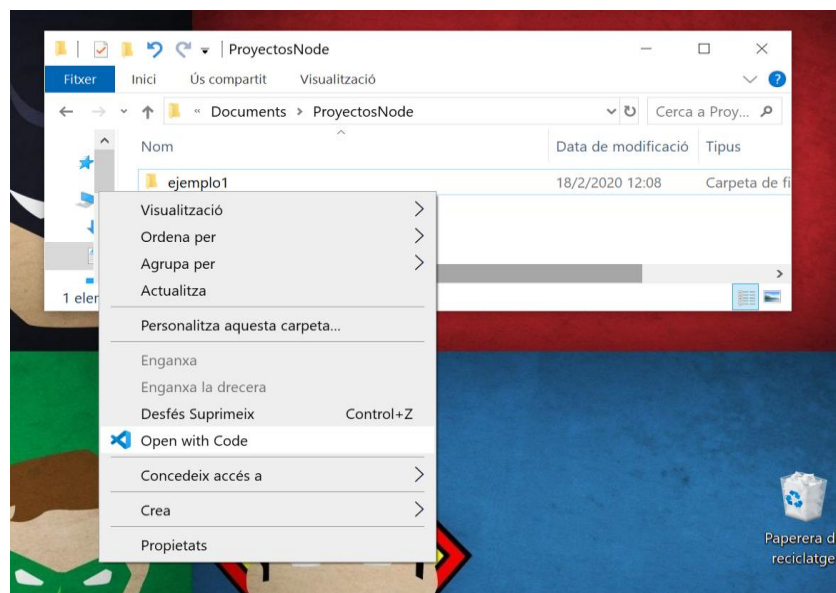
1. Cree una carpeta en la ubicación que desee, asígnele un nombre sin espacios en blanco.
2. Luego acceda a la carpeta desde su consola o terminal y ejecute el siguiente comando: **npm init**. El comando anterior genera un archivo package.json personalizado para cada proyecto de node.

Ejemplo de proyecto con node

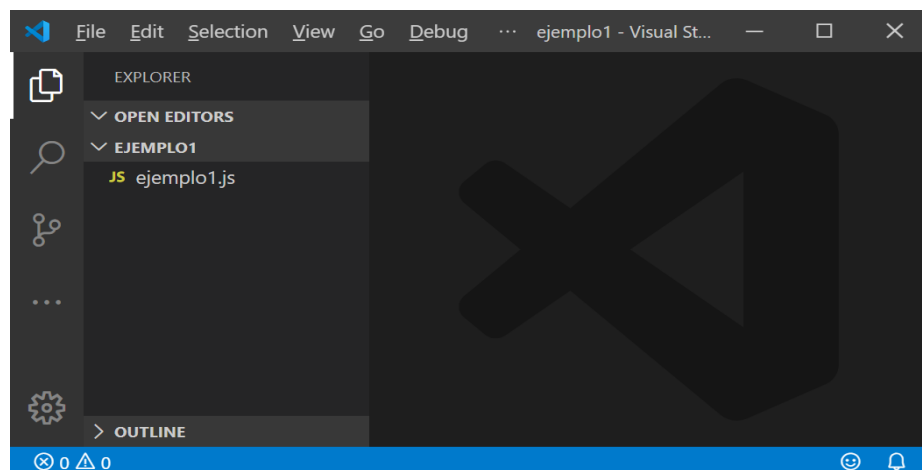
Vamos a crear un proyecto con node utilizando un archivo example1.js. Primero creamos un directorio llamado ejemplo1 y copiamos el archivo ejemplo1.js:



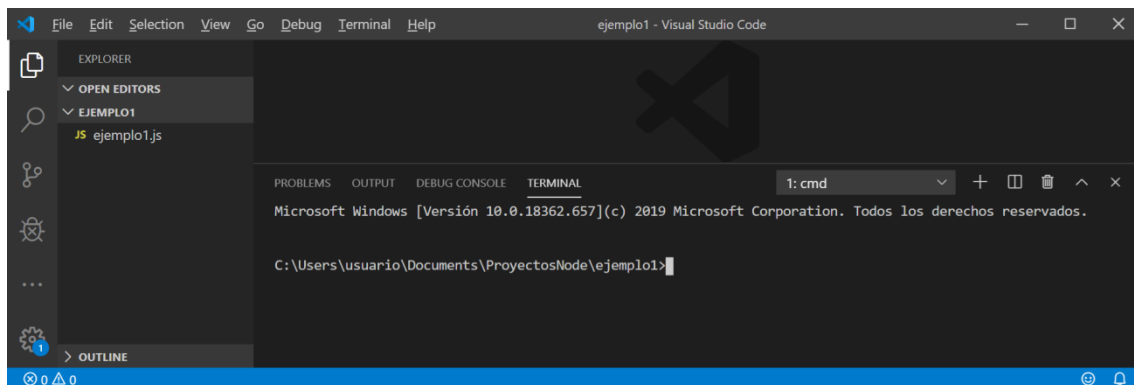
Abra la carpeta con el código de Visual Studio haciendo clic derecho en la carpeta ejemplo1 y seleccionando la opción para abrir con code:



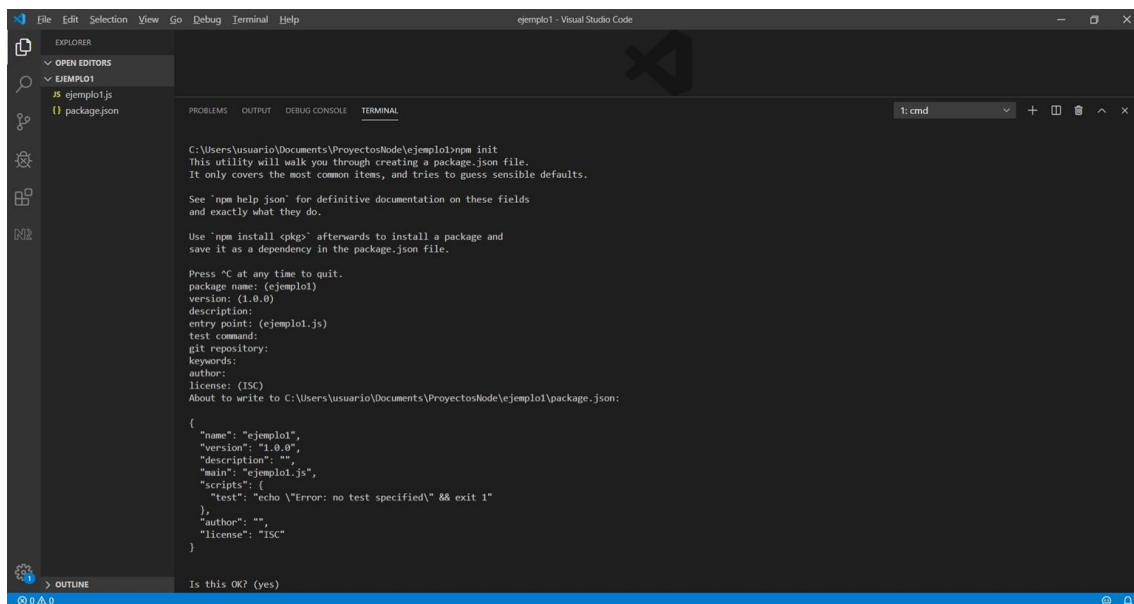
Visual studio code se abrirá con el contenido de esa carpeta:



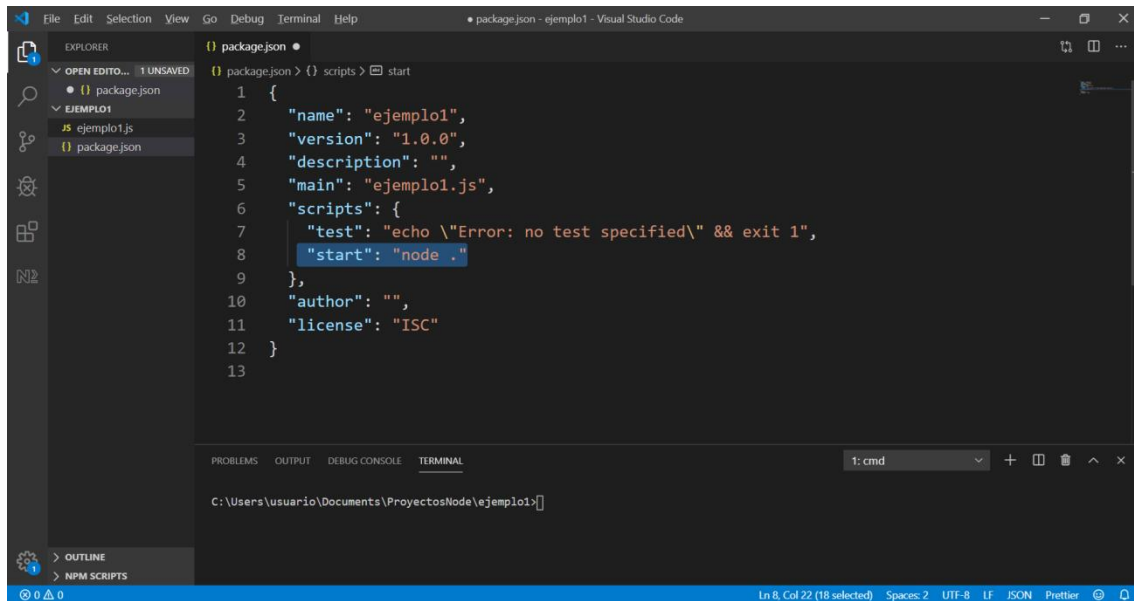
Para ejecutar npm podemos hacerlo desde la línea de comandos (**cmd**, no powershell porque dará error) habitual o usando el código. Para desplegarlo, haz "CTRL + ñ" y selecciona en la flecha **command prompt (cmd)**:



Con el comando **npm init** crearemos el proyecto, lo que implica generar un archivo **package.json** que define nuestro proyecto. Al ejecutar npm init desde nuestra carpeta de proyecto (en este caso, el ejemplo 1), podemos dejar todas las opciones predeterminadas presionando enter:



Vamos a agregar una línea a package.json para ejecutar el proyecto usando npm:

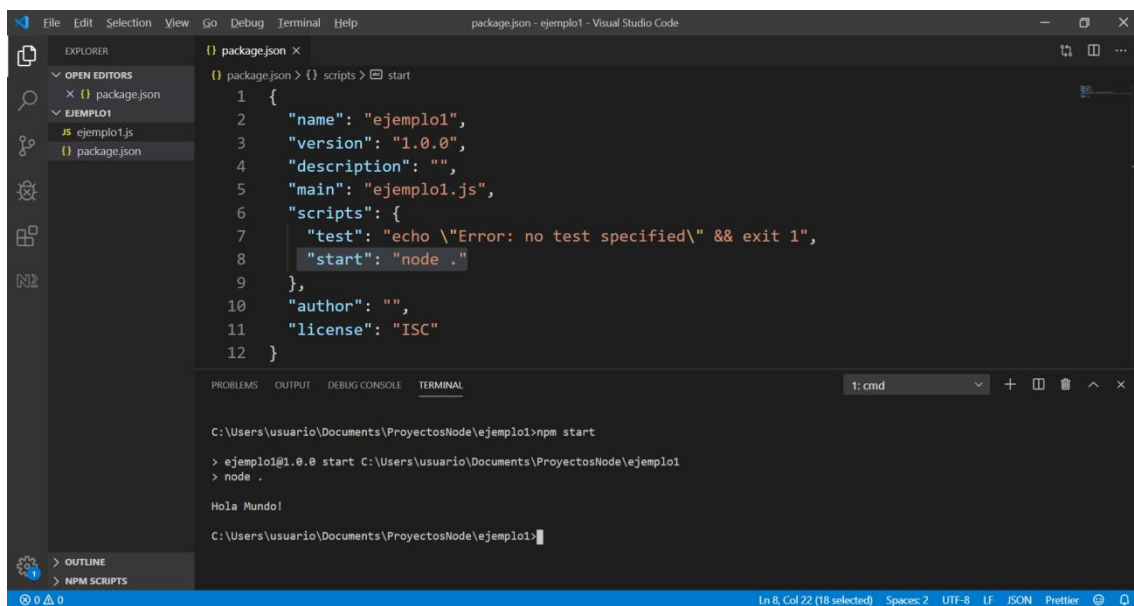


```
1 {
2   "name": "ejemplo1",
3   "version": "1.0.0",
4   "description": "",
5   "main": "ejemplo1.js",
6   "scripts": {
7     "test": "echo \\\"Error: no test specified\\\" && exit 1",
8     "start": "node ."
9   },
10  "author": "",
11  "license": "ISC"
12 }
13
```

Terminal output:

```
C:\Users\usuario\Documents\ProyectosNode\ejemplo1>
```

Guardamos el contenido de package.json, ahora podemos ejecutar el proyecto usando **npm start**:



```
C:\Users\usuario\Documents\ProyectosNode\ejemplo1>npm start
> ejemplo1@1.0.0 start C:\Users\usuario\Documents\ProyectosNode\ejemplo1
> node .

Hola Mundo!
```

Terminal output:

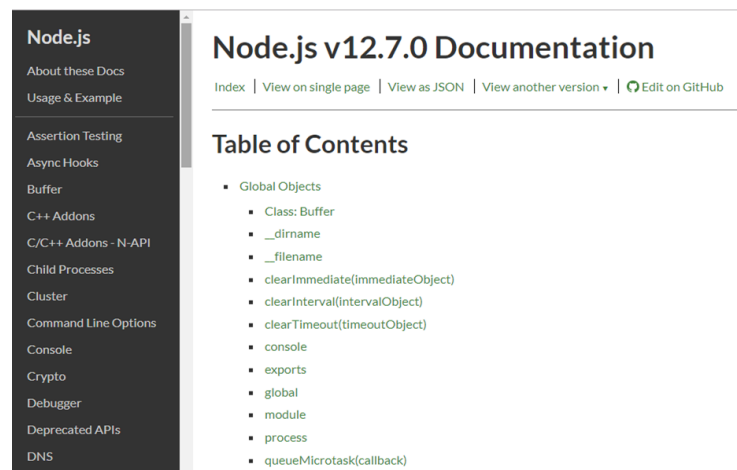
```
C:\Users\usuario\Documents\ProyectosNode\ejemplo1>
```

Lo que hace npm start es ejecutar el comando que hemos puesto en la línea **start** del package.json, usando como archivo javascript el especificado en la línea **main** del package.json.

3. El objeto global

Una vez que entendemos cómo Node.js permite ejecutar JavaScript fuera del navegador, es importante notar que el entorno de ejecución también cambia.

El objeto inicial de la jerarquía en Javascript dentro de un **navegador** sería la ventana, el **objeto window**. Sin embargo, en **node** no estamos dentro del navegador, y el **objeto global** es solo eso, el objeto global, y se llama global, aunque no es necesario escribirlo. A través de él podemos acceder a una serie de métodos y propiedades como podemos ver en la página de node (<https://nodejs.org/api/globals.html>).



The screenshot shows the Node.js v12.7.0 Documentation page. On the left is a dark sidebar with a list of links including 'About these Docs', 'Usage & Example', 'Assertion Testing', 'Async Hooks', 'Buffer', 'C++ Addons', 'C/C++ Addons - N-API', 'Child Processes', 'Cluster', 'Command Line Options', 'Console', 'Crypto', 'Debugger', 'Deprecated APIs', and 'DNS'. The main content area is titled 'Node.js v12.7.0 Documentation' and includes links for 'Index', 'View on single page', 'View as JSON', 'View another version', and 'Edit on GitHub'. Below the title is a 'Table of Contents' section with a list of 'Global Objects' including: 'Class: Buffer', 'dirname', 'filename', 'clearImmediate(immediateObject)', 'clearInterval(intervalObject)', 'clearTimeout(timeoutObject)', 'console', 'exports', 'global', 'module', 'process', and 'queueMicrotask(callback)'.

3.1. Funciones

Ya hemos usado el método console, ahora vamos a usar otras funciones globales que nos da Node.js.

Por ejemplo, **setTimeout** nos permite ejecutar una función **una sola vez** después de que transcurra un tiempo determinado. Es como poner un despertador: esperamos unos segundos y entonces se ejecuta el código indicado.

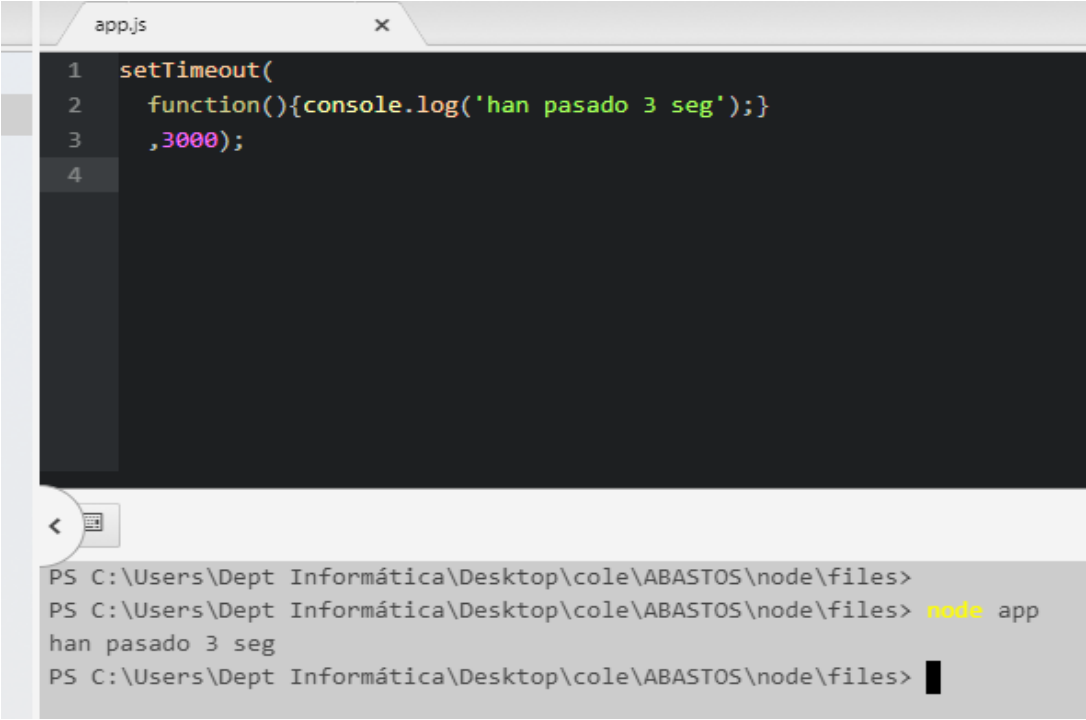
setTimeout(callback, delay[, ...args])

Added in: v0.0.1

- **callback** `<Function>` The function to call when the timer elapses.
- **delay** `<number>` The number of milliseconds to wait before calling the **callback**.
- **...args** `<any>` Optional arguments to pass when the **callback** is called.
- Returns: `<Timeout>` for use with `clearTimeout()`

Schedules execution of a one-time **callback** after **delay** milliseconds.

Ejemplo:



The screenshot shows a code editor window titled 'app.js' with the following JavaScript code:

```
1 setTimeout(  
2   function(){console.log('han pasado 3 seg');}  
3   ,3000);  
4
```

Below the code editor is a terminal window showing the command prompt output:

```
PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files>  
PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files> node app  
han pasado 3 seg  
PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files>
```

En cambio, **setInterval** hace que una función se ejecute **de manera repetida cada cierto intervalo de tiempo**, como un reloj que suena cada pocos segundos hasta que lo paramos.

setInterval(callback, delay[, ...args])

Added in: v0.0.1

- **callback** `<Function>` The function to call when the timer elapses.
- **delay** `<number>` The number of milliseconds to wait before calling the **callback**.
- **...args** `<any>` Optional arguments to pass when the **callback** is called.
- Returns: `<Timeout>` for use with `clearInterval()`

Schedules repeated execution of **callback** every **delay** milliseconds.

Ejemplo:

```
1
2
3 var time=0;
4 setInterval(
5     function(){
6         time += 2;
7         console.log('han pasado '+time+' seg');}
8     ,2000);
9
```

PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files> node app

han pasado 2 seg

han pasado 4 seg

han pasado 6 seg

han pasado 8 seg

han pasado 10 seg

PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files> █

Paramos el proceso con Ctrl + C en la terminal.

Para hacerlo parar de acuerdo a una condición utilizamos **clearInterval**:

```
2
3 var time=0;
4 var timer= setInterval(
5     function(){
6         time += 2;
7         console.log('han pasado '+time+' seg');
8         if (time>5){
9             clearInterval(timer);
10        }
11    }
12    ,2000);
```

PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files> node a

han pasado 2 seg

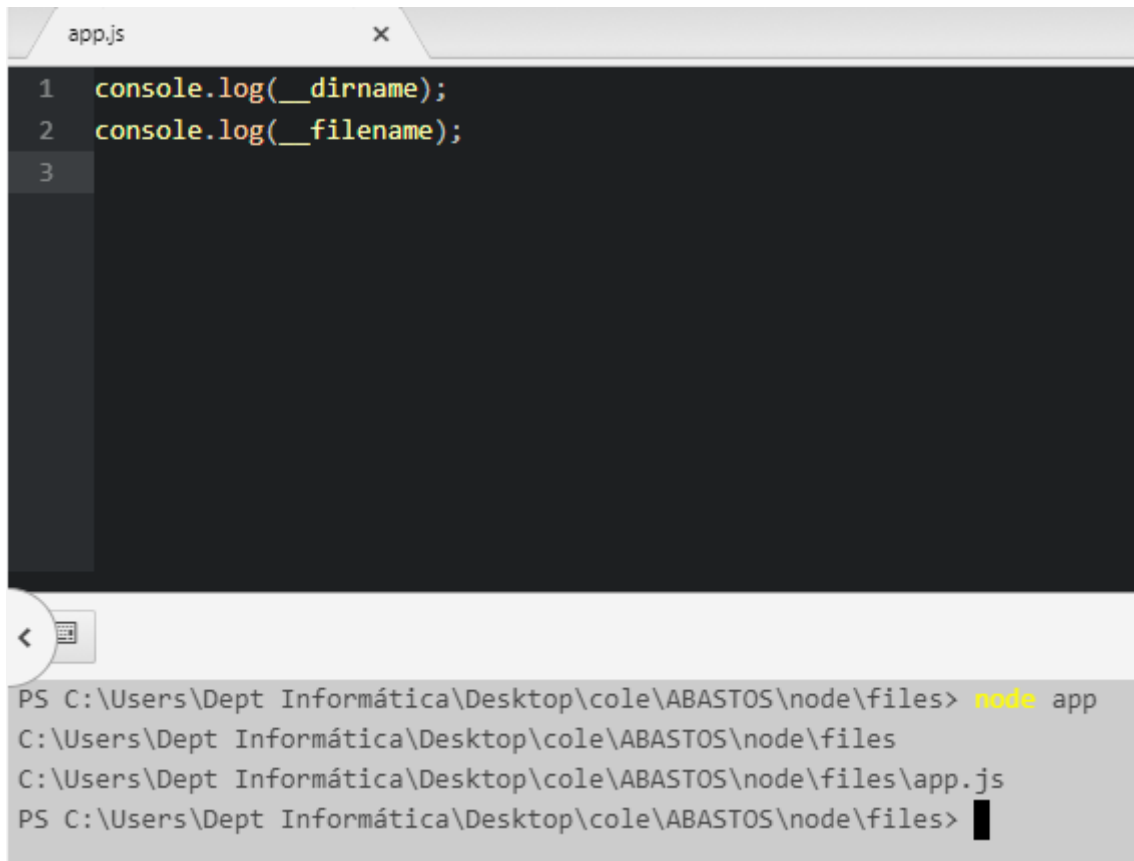
han pasado 4 seg

han pasado 6 seg

PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files> █

3.2 Propiedades

Además de estas funciones, Node.js también nos ofrece **propiedades globales** muy útiles. Entre ellas destacan `__filename` y `__dirname`, que nos indican el nombre del fichero actual y el directorio en el que se encuentra. Con estas propiedades podemos conocer siempre **dónde está ubicada nuestra aplicación**, algo fundamental cuando trabajamos con rutas, ficheros o módulos.



The image shows a code editor window titled 'app.js' with the following code:

```
1 console.log(__dirname);
2 console.log(__filename);
3
```

Below the code editor is a terminal window showing the execution of the script:

```
PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files> node app
C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files
C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files\app.js
PS C:\Users\Dept Informática\Desktop\cole\ABASTOS\node\files>
```